考核报告

——计算机科学与技术 263 班，刘宇浩

技术报告

第一关：找到入口

本任务其实比较简单，我们先用 ls 找到 key.txt，用 cat 查看，我们就知道了密钥是什么，也就是 MOON，接着看一下对 exam 执行一下 help，看到用法是
exam -e [密钥]
并且试着运行后会看出来 exam 会硬编码读取本目录下的 input，所以我们要做的很简单：用 mv 把 input.bin 改名为 input，运行后我们 ls 一下可以看到一个 output，把它改名成 flag 即可通关

第二关：两段信号

这一关比较系统地考察了下对 Linux 一些常用指令的掌握，
和之前一样，我们要先用—help 参数查看一下 exam 的用法
而我们知道，find 可以用于递归查找文件，而 grep 可以搜索特定内容，于是一切都简单了：我们先使用 find . -name “signal-A*.*”和 find . -name “signal-B*.*”，就可以很顺利地找到这两个文件

找到这两个文件后，就该把它们拼接起来，这时候我们就该使用重定向符了，由于 A 要在前面，我们就执行
cat ./archive/2026/signal-A.bin > input
cat ./inbox/signal-B.bin >> input
这样子 input 的准备工作就好了

接着我们要找到密钥，也就是含 ACCESS_TOKEN 的一个文件，这时候就需要 grep 和管道了，用 find 递归查找到所有文件，接着 grep 找到 ACCESS_TOKEN,用管道连接起来，就是
find . -name "*.*" | grep -r "ACCESS_TOKEN"
我们就能很简单知道，密钥是 STAR

然后就简单了，直接运行 exam，把生成的 output 直接改名 flag 即可结束。

第三关：读日志并修复输入

老规矩，进来先看一下 exam 的用法，可以发现还是一样要密钥，用 ls 总览一下后会

发现一个 key.txt，毫无疑问这个里面写的 NOVA 就是密钥，接下来我们按照提供的日志路径去查看，很容易发现根本就没有日志：很显然，这需要我们先执行一遍 exam 后才有，自然，我们需要把这个 input.bin 改成 input 后再运行一下，通过日志看一下说什么，

读数统计

本任务先判断 n 是否大于 10000，如果是则程序返回 1，防止出现越界问题，采取 getchar 函数消除前面的回车，防止读取失败卡死无法读取后面的正常数据。

读数统计

本任务先判断 n 是否大于 10000，如果是则程序返回 1，防止出现越界问题，采取 getchar 函数消除前面的回车，防止读取失败卡死无法读取后面的正常数据。

读数统计

本任务先判断 n 是否大于 10000，如果是则程序返回 1，防止出现越界问题，采取 getchar 函数消除前面的回车，防止读取失败卡死无法读取后面的正常数据。

读数统计

本任务先判断 n 是否大于 10000，如果是则程序返回 1，防止出现越界问题，采取 getchar 函数消除前面的回车，防止读取失败卡死无法读取后面的正常数据。

读数统计

本任务先判断 n 是否大于 10000，如果是则程序返回 1，防止出现越界问题，采取 getchar 函数消除前面的回车，防止读取失败卡死无法读取后面的正常数据。

读数统计

本任务先判断 n 是否大于 10000，如果是则程序返回 1，防止出现越界问题，采取 getchar 函数消除前面的回车，防止读取失败卡死无法读取后面的正常数据。

读数统计

本任务先判断 n 是否大于 10000，如果是则程序返回 1，防止出现越界问题，采取 getchar 函数消除前面的回车，防止读取失败卡死无法读取后面的正常数据。

读数统计

本任务先判断 n 是否大于 10000，如果是则程序返回 1，防止出现越界问题，采取 getchar 函数消除前面的回车，防止读取失败卡死无法读取后面的正常数据。

读数统计

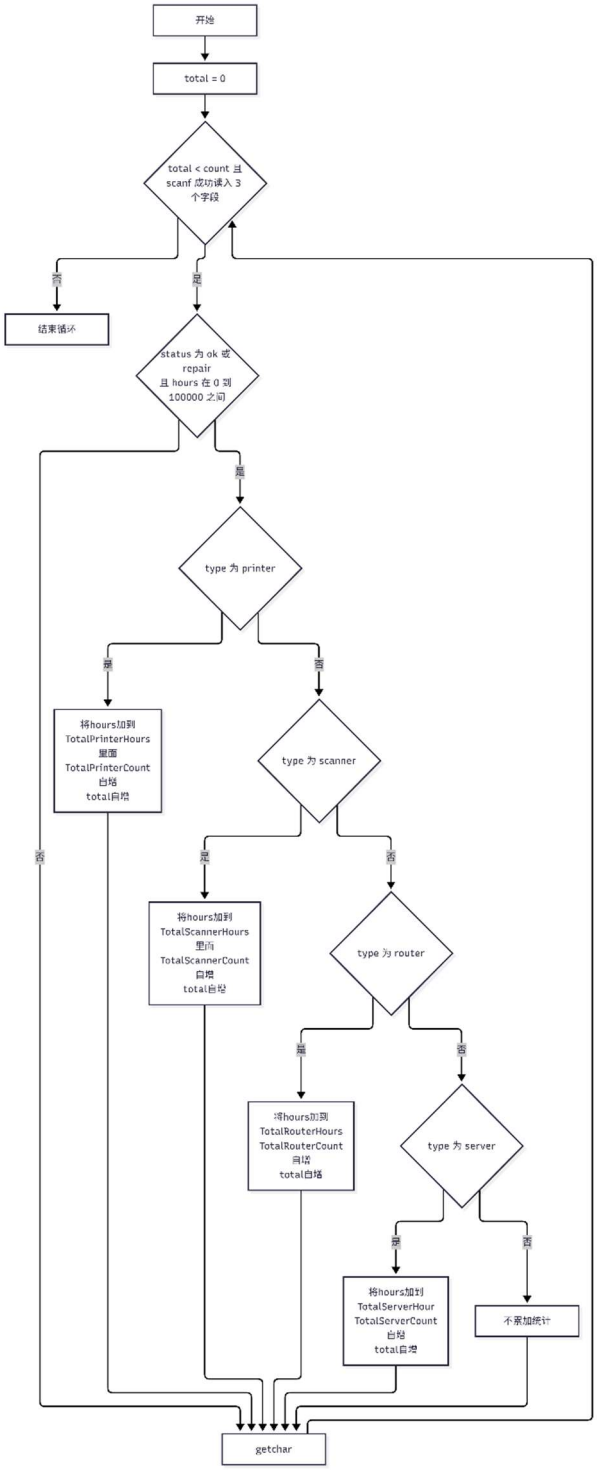
本任务先判断 n 是否大于 10000，如果是则程序返回 1，防止出现越界问题，采取 getchar 函数消除前面的回车，防止读取失败卡死无法读取后面的正常数据。

设备巡检记录

本任务先消除掉读取 count 后残留的换行符，避免后续 scanf 无法读取，接着定义八个变量，分别为 printer，scanner，router，server 这四个分别对应的总时长与次数，接着进入循环。

在循环里，我们先检验一下 scanf 是不是读到了这三个数据，要是读不到就脱离循环，同时我们使用 total 变量控制循环次数，如果在 scanf 还能读取的情况

下 total 还没达到输入的 count，那么就持续循环读取。（以下为循环流程图）



读取到数据后，我们先对照一下 status 是不是 ok 或 repair，以及 time 是不是在[0,100000]内，如果不是 total 不自增，跳入下一循环，是的话继续看前面

type 是啥，并把 time 加入对应的总时长，次数加 1，total 加 1，没有继续下一循环，每次进入下一个循环都先用 getchar 清除一下换行符。

读取完所有的数据就该判断 busiest 是哪个了，我们先假定 busiest 是 1(不初始化为 0 的原因是为了防止后面全都匹配不到导致无 busiest，当然，我们还有另一种办法，将 TempTime 初始化为-1 就可以了)，并定义一个 TempTime 变量初始化为 0，用于比较，我们首先对比一下 Temptime 和 printer 的时间 TotalPrinterHours，如果前者更多就保持 busiest 和 TempTime 不变，如果后者更多就把 TempTime 和 busiest 和后面那个靠齐，以此推下去将 4 个按顺序对比，就可以得到时长最多且最靠前的。

固定窗口读数工具

本任务的处理十分简单，对 i 进行取 CAPCITY 的余（我个人认为，这里的 8 不能写死，定义这个 CAPCITY 是为了实现解耦，便于根据需求修改所需窗口大小)，使得当 i 达到 CAPCITY 后可以回绕从下标 0 存储，覆盖掉最早的一个。

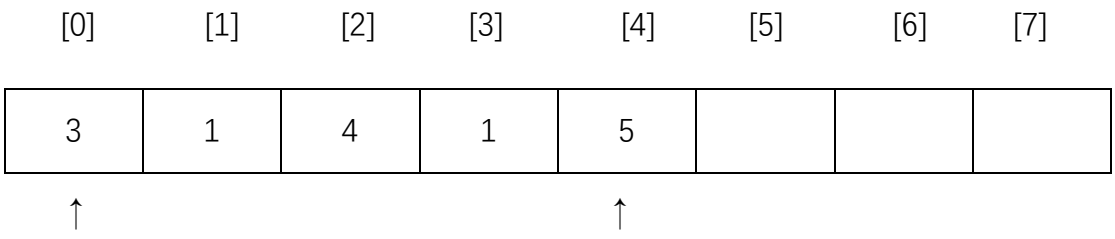
至于 keep 的控制，很简单，我们设计一个 if，当 keep 没达到 8 就自增，达到了 8 就不自增

同时，我们定义一个 NewPointer 和 OldPoint 分别用于指向最新的和最旧的数

据，在 i 未超过 8 时，keep 正常增加，而当 keep 达到 8 后，keep 不再增加，同时 OldPointer 指向对 i+1 取 8 的余数，这样 OldPointer 始终指向 NewPointer 前面的一个，最快被覆盖的就是最早的那个

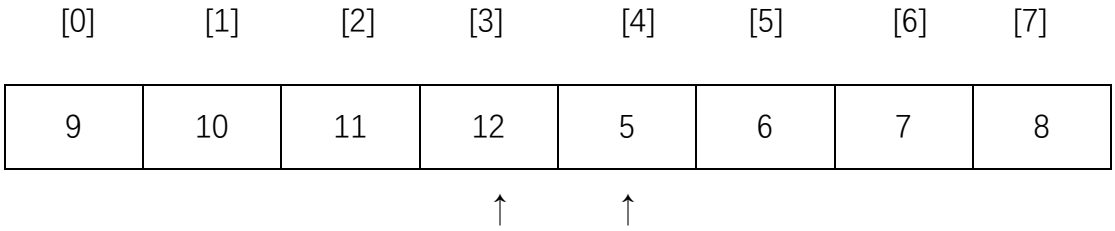
以题目里的样例为例

样例 1：(3，1，4，1，5)



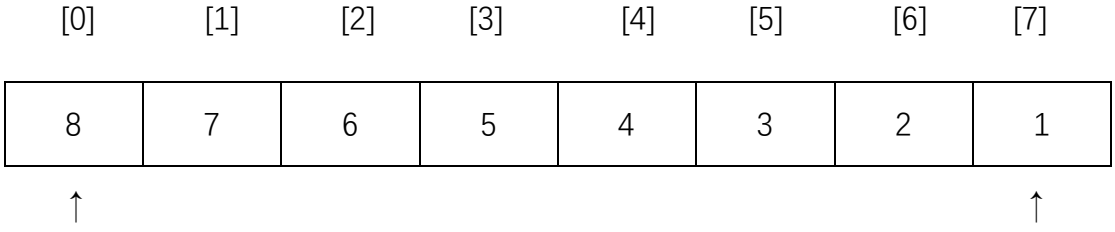
OldPointer NewPointer

样例 2：(1，2，3，4，5，6，7，8，9，10，11，12)



NewPointer OldPointer

样例 3：(8，7，6，5，4，3，2，1)



OldPointer NewPointer

多文件统计工具

这个任务要求修复由多个文件组成的文件统计工具，在 main()里，使用的格式是：

void print_report(int count, long long total, int smallest, int largest)

而在 format.c 和 format.c 中，参数顺序颠倒，我们要做的比较简单，就是对照 main()里的调用类型，反过来修改 format.h 和 format.c 里参数列表里的类型，使之匹配

除了这个，我们还需要修改 CMakeLists.txt，给 add_executable 里面加上 format.c，并且加上这两句：

set(CMAKE_C_STANDARD 11)

set(CMAKE_C_STANDARD_REQUIRED ON)

第一行指定使用 C11 标准，第二行执行强制要求使用这个标准。

加上这两行后，整个任务就完成了。

自然，不要忘记把文件从 c 文件夹里拿出来。

感悟报告

读数统计

在写这读数统计时，我最开始采取的方案是采用`fgets`读取一整行，再通过`sscanf`依次读取数据，由于`sccanf`无法自动移动，需要一个巨大的字符数组去接收，而且需要防止缓存区溢出，而且会因未知原因造成输出与预期不符合：

```
char buffer[40000];

getchar();

fgets(buffer,sizeof(buffer),stdin);

//printf("%s",buffer);

int max = -1000,min = 1000,count = 0,sum=0;

int temp = 0, offset = 0 ,size = 0;

while (count < n && sscanf(buffer+offset,"%d%n",&temp, &size) == 1)

{

    //由于 sscanf 无法自己移动所以需要我们用一个 size 来表示读取了多少个字符，并将其加到 offset 上

    offset += size;

    count++;

    sum += temp;

    if (temp>max)

    {

        max = temp;

    }

    if (temp<min)

    {

        min = temp;

    }

}
```

所以在最后和 AI 讨论后改为采取`scanf`读取，更加简单（当然，前者一般在实际业务

使用更多，但对我而言难度比较高，所以先采用`scanf`方案）

设备巡检记录

这一关也是问题比较多，原本我以为用 scanf 只要读取成功就一定返回 1，但实际情况是输入足够数量的数据没有结束，经过询问 AI，得知 scanf 的返回值取决于成功读取了几个数据，所以在此处，scanf 的返回值应该为 3。

还有，不知道是不是是太久没摸 C 语言了，我闹了两个笑话：一个是在 switch 里使用字符串，但其实 switch 里面只能用 int，另一个是在 C 语言里，字符串的对比不能用==，要用 strcmp()函数，当然，或许也可能是我以前做的程序很少出现和字符串打交道的情况，以前写 switch case 时，经常用枚举来方便自己阅读

除了这个，在 busiest 上，前面技术报告里我也说了，本来是将 busiest 初始化为 0 的，根据对比时间来确定 busiest 是哪一个(要换我以前，估计又要开始写枚举了)，当然，由于查出了 bug，AI 建议我把代码改成 busiest 初始化为 1，但事后我又想到另一种方法，也就是把 TempTime 初始化为-1，这样也可以防止这种全 0 的情况，不过由于已经提交了也不太好改了。

固定窗口读数工具

我敢说这是整个 C/C++方向对我最友好的一关：除了最开始的回绕不明白什么意思，不过用 AI 查了一下就知道了这个是什么意思，一开始我想到的是队列，但是我们又不可能从头写一个队列：太费时间了，所以，我采用两个指针(说白了就是两个记录下标的变量，不是那种记录变量地址的指针变量)，一个指向最早的，一个指向最新的

不过在写关于这两个变量怎么动的时候，我又面临一个问题：在输入数据未达到 8 时，OldPointer 不需要移动，而到 8 以后我们就要移动 OldPointer，这个我后来想到了 keep，利用 keep 是否小于 8 的两种情况来分别处理 OldPointer 的不同移动方法，

原本关于 OldPointer 的移动方法我也十分头疼，后来我想到，最早进来的也会最先被覆盖，那么哪个最快被覆盖哪个就是最先进来的，也就是 i+1 的取余。所以很快就完成了，by the way，这估计是最快完成的一关。

多文件统计工具

关于多文件编程，虽然以前自己做程序时，也常常把各功能块分离，便于自己查找，不过由于以前比较依赖 IDE，所以关于怎么把这几个文件放一起用的 CMake 我不太了解，自然又去补了一番课，在弄这个 CMakeLists.txt 时，我也闹了不少笑话，照抄 hints.txt 时，忘记把_CXX_改成_C_了，当然，由于我不懂逆向，所以我也不知道 CMake 是以一个什么样的心情看着这个前面说语言是 C 语言后面又说要 CXX 11 的标准的文件的(狗头保命)。

由于那天做这个时我前面在晚自习上刚完成固定窗口读数工具，所以我一股子劲地直接来对付这一关，所以甚至搞出了“把 largest 的类型改成 long long”的闹剧，好在最后是修复好了，也发现了这个类型的不匹配，其实主要还是心焦了，直接看着 format.h 时就开始直接修改了，后来仔细检查 main 调用 print_report 时才注意到这个问题，从头调整了一遍，至于过没过关……我在这都排了两天队了都，我还是不知道我现在是不是确实没有问题了。